

# Minimal Comparison of Octagonal Abstract Domains

Kenny Ballou<sup>✉</sup>

California State University San Marcos  
San Marcos, USA  
kballou@csusm.edu

Elena Sherman

Boise State University  
Boise, USA  
elenasherman@boisestate.edu

## Abstract

Numerical abstract domains vary in their expressiveness; more expressive domains like Zones yield more precise invariants than Intervals. A comprehensive approach to selecting abstract domains is a minimal comparison of abstract states. However, to be effective, it requires abstract states to be free of spurious constraints. While previous work developed spurious constraint elimination for Zones, this work introduces a novel algorithm for eliminating such constraints for Octagons.

We evaluate our approach by comparing the precision of 6,930 invariants from different abstract domains. Our results show that the minimal comparison reclassifies many invariants as equivalent, thus, reducing the impact of Octagons' expressiveness on invariant precision.

**CCS Concepts:** • **Theory of computation** → *Program analysis; Program verification; Invariants.*

**Keywords:** Octagons, Domain Comparison, Weakly-Relational Abstract Domains

## ACM Reference Format:

Kenny Ballou and Elena Sherman. 2026. Minimal Comparison of Octagonal Abstract Domains. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Numerical and Symbolic Abstract Domains (NSAD '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3840563.3843710>

## 1 Introduction

The tradeoff between expressiveness and efficiency of different numerical abstract domains such as Zones [25], Octagons [27] or Intervals, requires an understanding of the impact of precision loss when choosing a more efficient, but less expressive abstract domain. While related work [1, 7, 18, 24] establish relations between program structural characteristics and an appropriate abstract domain choice, this information alone is insufficient to comprehensively compare domains.

Traditionally comparing two abstract domains,  $D_1$  and  $D_2$ , where the former is more precise than the latter, involves

comparing their invariants  $I_1$  and  $I_2$  at the same program locations. If the invariants of  $D_1$  dominate in precision then one concludes a high impact on precision gain if  $D_1$  is selected. Other works [8, 16, 28], create a distance metric to compare abstract states. However, in the context of data-flow analysis (DFA), previous work [3, 4] show that a novel minimal comparison technique provides more insight into the precision gain or loss when choosing  $D_1$  or  $D_2$ , respectively.

The minimal comparison leverages the fact that at each computational step, DFA applies incremental changes to an abstract state, i.e., a transfer function only modifies a part of an abstract state. While other techniques use incrementality to improve performance [2, 13, 20], the line of work on minimal comparison uses incremental updates to identify the changed portion of the abstract state and hence, compare only the updated parts of the resulting invariants. This eliminates cases when an improvement in precision in one statement propagates through the entire computation. For example, in this sequence of statements  $1: x = y + 2$ ,  $2: z = 2$ ,  $3: w = 4 - z$ , using Zones increases precision after statement 1: but for the other two statements, Zones computes the same invariants as Intervals for  $z$  and  $w$  components of abstract states. Yet, comparison of the entire state results in Zones being globally more precise.

To effectively apply the minimal change algorithm [3], it is imperative to remove spurious dependencies between variables in a fully closed form of an abstract state. While previous work developed such an algorithm for Zones [3, 4], it cannot, however, be directly applied to Octagons because of the substantial differences in their state representations. Thus, in this work we present a novel algorithm for removing spurious dependencies for fully closed Octagon abstract states, which we describe in Section 3. Equipped with this new algorithm, we perform extensive empirical evaluation on how comparing minimal changes of invariants impacts comparison of abstract domains within the context of DFA, Section 4. We discuss related works in Section 5 and finally conclude with future works in Section 6.

## 2 Background

### 2.1 Zones

The Zones domain [25] represent predicates via a strict unit difference formula, extending the intervals domains with the template formula  $x - y \leq c$ , where  $x$  and  $y$  are program variables, and  $c$  is a numerical constant from one of  $\{\mathbb{Z}, \mathbb{Q}, \mathbb{R}\}$ . Interval valued constraints are encoded by adding a special



This work is licensed under a Creative Commons Attribution 4.0 International License.

NSAD '26, Oakland, CA, USA

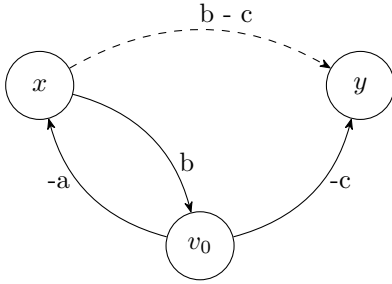
© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2933-1/2026/10

<https://doi.org/10.1145/3840563.3843710>

variable, typically denoted  $v_0$ , such that its value is always zero. Thus,  $x - v_0 \leq b \implies x - 0 \leq b \implies x \leq b$  and  $v_0 - x \leq -a \implies 0 - x \leq -a \implies x \geq a$  are sufficient to encode the interval for  $x$ :  $x \mapsto [a, b]$ . Figure 1 depicts the directed, weighted graph representation of a Zone state for the interval constraints  $x \mapsto to[a, b] \wedge y \mapsto to[c, +\infty)$ . The nodes of the graph represent the constraint variables, and the edge weight denotes the difference constraint between the variables. The direction of the edge represents the ordering of the formula. Moreover, the graphical representation has an isomorphic representation as a difference bounded matrix (DBM).

The usefulness of the directed, weighted graph representation is in using shortest path closure algorithms, e.g., Floyd-Warshall [11], to infer constraints between variables and to otherwise normalize the possible constraint system. As such, Figure 1 has a dashed edge between  $x$  and  $y$  representing the constraint  $x - y \leq b - c$ , an inferred constraint via such a closure operation.



**Figure 1.** Directed weighted graph representation of unit-difference constraints of a Zone abstract state.

## 2.2 Octagons

The Octagons domain [27]<sup>1</sup> extends the unit difference of Zones by encoding unit constraints over program variables in the forms  $\pm x \pm y \leq c$  and  $\pm x \leq 2c$ . The symbols  $x, y$  represent program variables, and  $c$  is a numerical constant<sup>2</sup>. To efficiently manipulate such constraints, researchers proposed an encoding [9, 13, 27] where each program variable translates to a pair of constraint nodes, denoted here as  $x^+$  and  $y^+$ , which can similarly be expressed as a directed, weighted graph using difference constraints. The standard Octagon encoding rules are shown in Figure 2. Notice, while an Octagon uses DBMs to represent its constraints, it necessarily doubles the graph nodes to encode the plurality of each variable, e.g.,  $x^\pm$ .

Consider the code snippet: `if (y >= a){ x = b; . . . }`, for example, where  $a, b$  are constants and  $x, y$  are program

$$\begin{aligned} x - y \leq c &\rightsquigarrow x^+ - y^+ \leq c \wedge y^- - x^- \leq c \\ x + y \leq c &\rightsquigarrow x^+ - y^- \leq c \wedge y^+ - x^- \leq c \\ -x - y \leq c &\rightsquigarrow x^- - y^+ \leq c \wedge y^- - x^+ \leq c \\ x \leq c &\rightsquigarrow x^+ - x^- \leq 2c \\ x \geq c &\rightsquigarrow x^- - x^+ \leq -2c \end{aligned}$$

**Figure 2.** Standard encoding rules for converting program variable space predicates and converting them to octagonal variable space.

variables. The graph and DBM in Figure 3 shows the octagonal encoding of the state on the true branch, i.e.,  $y \geq a \rightsquigarrow y^- - y^+ \leq -2a$  and  $x = b \rightsquigarrow x^+ - x^- \leq 2b \wedge x^- - x^+ \leq -2b$ . After processing  $x = b$ , DFA modifies the state in Figure 3a, with the newly added constraints depicted in the highlighted box. We collect these newly added edges into a set,  $de$ , which represents the changed edges as it relates to the abstract state. In this case, the set  $de$  is equal to the following  $\{(x^+, x^-, 2b), (x^-, x^+, -2b)\}$ , which is equivalent to the logical expression,  $x^+ - x^- \leq 2b \wedge x^- - x^+ \leq -2b$ .

## 2.3 Symbolic Predicates Abstract Domain

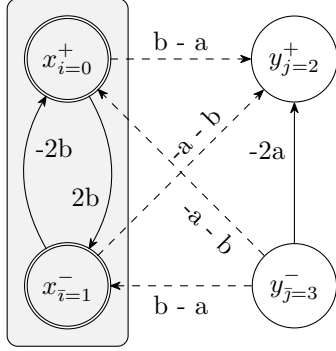
For evaluating our comparison, we use a so-called symbolic predicate domain to represent an instance of an incomparable domain. The Symbolic Predicates abstract domain extends the well-known Predicates domain with arbitrary inequality formula. These inequalities are automatically inferred through the program's transfer functions when performing Predicate analysis [30]. Furthermore, since these inequalities are arbitrary, they can add exact, relational information to the predetermined set of predicates. For example, using the Sign domain, interpreting the expression  $y = -2x$ , with an incoming state of  $x \rightarrow \{-\} \wedge y \rightarrow \top$ , becomes  $x \rightarrow \{-\} \wedge y \rightarrow \{+\}$ . Augmenting with symbolic information, the expression after computing its transfer function becomes  $x \rightarrow \{-\} \wedge y \rightarrow \{+\} \wedge y = -2x$ , a precise relational representation of the assignment. We simply use this domain as an instantiation of an incomparable domain to Octagons.

## 2.4 Additional Notations

While we use some common notation throughout this work, we define some of the notation for clarity. When using the matrix notation for DBMs, we often use index variables  $i, j, \bar{i}, \bar{j}$ . Previous work has used the latter two, to represent the “negative” dual of the former [9]. That is, we denote variable pairs within the octagonal encoding using  $i$  and  $\bar{i}$  for each real program variable such that  $i \oplus \bar{i} \equiv 1$ , where  $\oplus$  is the exclusive or (XOR) operator. Furthermore, we can recover

<sup>1</sup>Also known as the unit two variables per inequality (uTVPI) domain.

<sup>2</sup>Our work focuses on integer constraints, but should easily extend to  $\mathbb{Q}$  and  $\mathbb{R}$  since integers require additional care the others do not.



(a) Fully closed graphical representation of an octagonal state.

$$\begin{array}{c}
 x^+ \\
 x^- \\
 y^+ \\
 y^-
 \end{array}
 \begin{bmatrix}
 x_{i=0}^+ & x_{i=1}^- & y_{j=2}^+ & y_{j=3}^- \\
 0 & 2b & b-a & +\infty \\
 2b & 0 & -a-b & +\infty \\
 +\infty & +\infty & 0 & +\infty \\
 -a-b & b-a & -2a & 0
 \end{bmatrix}$$

(b) Isomorphic difference bounded matrix (DBM) representation of the same octagonal state.

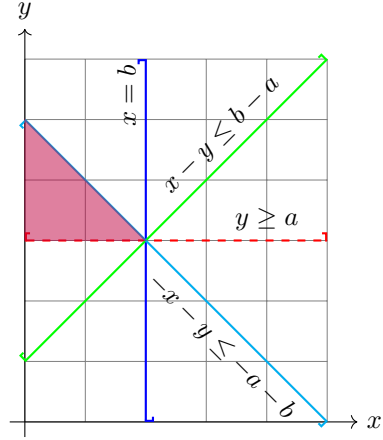
**Figure 3.** Fully, strongly closed Octagons represented as a weighted, directed graphs and its equivalent representation as a difference bounded matrix (DBM). The state contains  $y \geq a \wedge x = b$  as well as inferred constraints  $x - y \leq b + a \wedge -x - y \leq a - b$ . The double circles denote changed variables, dashed edges denote inferred edges through closure.

any index's pair by flipping operands:  $i \oplus 1 \equiv \bar{i}$ . Thus, an edge between  $i$  and  $\bar{i}$  represents the interval edges between a variable within an Octagon constraint system. Moreover, these notations are equivalent to the even/odd  $2i_x$  and  $2i_x + 1$ , encoding described by Miné [27].

## 2.5 Spurious Constraints

The critical step of finding a minimal state change is to precisely identify all variables,  $\Delta vars$ , affected by the newly added constraint  $de$ . To do so, our previous work [3] explores the neighborhood of the changed variables  $dv$  of  $de$  within the graph [3]. In our running example, the gray box includes all affected variables  $\Delta vars$ , which in this case is only the changed variables  $dv$  since there are no edges connecting them to  $y^+$  and  $y^-$ .

However, when an abstract state is in a canonical, fully closed form, those variables become connected through implicit constraints shown as dashed edges in Figure 3a, hence marking the variables  $y^\pm$  as changed too. We call these edges *spurious* since they do not further constrain the bounded region, but also decrease the precision of finding a minimal state change. To further illustrate this point, Figure 4 shows the feasible region for our running example constraints. Notice, however, that each of the inferred relational constraints



**Figure 4.** A projection of the feasible region from our running example constraints, defined in Figure 3

**Algorithm 1** Identify minimally changed variables within an Octagon.

**Require:**  $M$  is a strongly closed Octagonal DBM and  $de$  be the set of changed vertices within  $M$ .

```

1:  $dv \leftarrow \{ \}$ 
2: for  $(i, j) \in de$  do
3:   if  $i \equiv j \vee \text{IsIntervalValued}(i, j)$  then
4:      $dv \leftarrow dv \cup \{i, j\}$ 
5:   else
6:      $dv \leftarrow dv \cup \{i\}$ 
7:   end if
8: end for
9:  $\Delta vars \leftarrow dv$ 
10: for  $v \in dv$  do
11:    $\Delta vars \leftarrow \Delta vars \cup \{i \mid i \in \{0 \dots 2N\} \wedge M_{v,i} < +\infty\}$ 
    {Select forward reachable variables}
12:    $\Delta vars \leftarrow \Delta vars \cup \{i \mid i \in \{0 \dots 2N\} \wedge M_{i,v} < +\infty\}$ 
    {Select backward reachable variables}
13: end for
14:
15: return  $\Delta vars$ 

```

connecting  $x$  and  $y$  are purely tangential to the actual available values of  $x$  and  $y$ .

Since the fully closed form is imperative for running DFA, e.g., for comparing states, we propose a new algorithm for removing such spurious constraints in Octagon abstract states, which we discuss in the next section.

## 3 Minimal Comparison

Algorithm 1 extends the algorithm from previous work [3] to identify minimally changed variables  $\Delta vars$  when an abstract state  $M'$  is updated to a new state  $M$  after processing

**Algorithm 2** Constraint reduction algorithm for Octagons

---

**Require:**  $M \equiv C(M)$ :  $M$  is a fully closed Octagon DBM  
**Ensure:**  $(C \circ \mathcal{R})(M) \equiv M$ : The closure of the reduction is equivalent to  $M$ .

```

1: for all  $i \in \{0, \dots, 2N\}$  do
2:   for all  $j \in \{0, \dots, 2N\}$  do
3:     if  $i \equiv j \vee \text{IsIntervalValued}(i, j)$  then
4:       continue
5:     end if
6:     if  $M_{i,j} \geq \frac{M_{i,\bar{i}} + M_{\bar{j},j}}{2}$  then
7:        $M_{i,j} \leftarrow +\infty$ 
8:     end if
9:   end for
10: end for

```

---

a set of constraints  $de$ , where each constraint contains a triple of variables  $(i, j, w)$ . After  $\Delta$ vars are identified, then only constraints containing those variables are selected for comparison. Thus, the smaller the set of changed variables that Algorithm 1 returns, the more precise the comparison.

The algorithm takes the updated fully closed state encoded as a DBM  $M$  and the constraint set  $de$  associated with the update, where  $de$  is the set of incoming, modified constraints. First, on lines 2–8, the algorithm iterates over each constraint to determine whether  $i$  or  $j$  indeed changed. The `IsIntervalValued` predicate function simply returns true if the condition  $i \oplus j \stackrel{?}{=} 1$  succeeds. Next, on lines 10–13, the algorithm finds variables that are affected by the directly updated variables  $dv$ . Since  $M$  is fully closed, it requires only one “forward” step and one “backward” step in the graph’s representation of  $M$ . Here  $N$  denotes the number of *program variables* and  $|M| = 2N \times 2N$ , as per standard Octagon encoding that doubles the variable space [27].

However, when applied to fully closed states, Algorithm 1 is ineffective in identifying minimal changes because such states contain many spurious constraints for variables that encode interval values, similar to  $x$  and  $y$  from our example in Figure 3. That is, as part of the closure operation and necessary operation, many variables become spuriously connected. In our example,  $x$  becomes spuriously coupled with  $y$  in the constraint  $x - y \leq b - a$ . Algorithm 1 identifies  $x$  and  $y$  as part of the  $\Delta$ vars set and uses both for comparison, instead of just  $x$ . Therefore, we need a technique which removes these *spurious* constraints in  $M$  before passing this DBM to Algorithm 1.

We define a *spurious constraint* as a redundant constraint coupling variables within a DBM  $M$ . A positive-negative pair of variables within the octagonal DBM represent the interval bounds of a program variable, e.g.,  $(x_i^+, x_i^-, 2b) \equiv x \leq b$ , where  $i \oplus \bar{i} \equiv 1$ ; dually for the other direction. Due to the

closure operations, pairs of variables become coupled in a constraint. Notably, this is a desired property of closure and weakly-relational domains since it may add more precise information. However, if a constraint is the result of only interval constraint closures, then we label it as *spurious*. Formally, a constraint  $M_{i,j}$  in an Octagon matrix  $M$  is spurious when the following two conditions hold:

1.  $i \neq j \wedge i \oplus j \neq 1$ , and
2.  $M_{i,j} \geq (M_{i,\bar{i}} + M_{\bar{j},j})/2$ .

The first condition ensures that it is not itself an interval constraint. For example, we do not consider constraints such as  $M_{i,\bar{i}}$ . The second condition checks that a constraint over two variables is less or equally constraining than its strong closure [27]. If this constraint is stronger, it is more restrictive than the interval values alone.

Our novel Algorithm 2 removes spurious constraints for fully closed Octagons states, i.e., it removes all such recoverable constraints. To do so, the algorithm iterates over all indices of the closed DBM  $M$ , lines 1–2, skipping interval valued constraints, line 3. Here, as before,  $N$  is the number of *program variables*. In lines 6–7, if the constraint is *spurious*, the algorithm sets its value to  $+\infty$ .

The runtime for Algorithm 2 is  $O(N^2)$ . Since the for loops are definite with increasing indices, assuredly the algorithm terminates. Furthermore, the correctness of the algorithm is given by its post-condition where all removed constraints are recoverable by reapplying (strong) closure.

## 4 Experiments and Results

To evaluate the effectiveness of the spurious constraint reduction algorithm for Octagons, we compared the precision of invariants, via logical implication, computed by DFA using Octagons against those from other abstract domains. To establish the baseline data, we compare entire invariants, i.e., full state comparison or Full Comparison (FC). Next, we minimally compare invariants using Minimal Neighbors (MN) by selecting only the constraints of changed variables by leveraging Algorithm 1. Lastly we apply MN on invariants that underwent the spurious constraint reduction as described in Algorithm 2 (MN + Reduction).

We divide experiments into comparisons of Octagons with Zones and Intervals, i.e, comparable numerical domains, and an incomparable, Symbolic Predicates, domain. Within the comparable experiments, we subdivide the comparisons into three categories again: full comparison, our baseline; minimized comparison, that is, only using Algorithm 1; and minimized plus reduction, Algorithm 1 + Algorithm 2.

The Symbolic Predicates domain [30] used in this study includes the following initial disjoint predicate elements:  $\{(-\infty, -5], (-5, -2], -1, 0, 1, [2, 5), [5, +\infty)\}$ . This predicate domain was derived from values identified from a previous empirical study [10]. While specialized predicates for each program would likely improve precision for the symbolic

**Table 1.** Intervals and Zones vs Octagons invariants comparison

| Technique       | $I \equiv O$ | $I \prec O$ | $Z \equiv O$ | $Z \prec O$ |
|-----------------|--------------|-------------|--------------|-------------|
| Full Comparison | 1523         | 5407        | 5138         | 1792        |
| MN              | 3894         | 3036        | 6183         | 747         |
| MN + Reduction  | 4439         | 2491        | 6527         | 403         |

**Table 2.** Octagons and Symbolic Predicates

| Technique  | $O \equiv P$ | $O \prec P$ | $O \succ P$ | $O \prec\!\succ P$ | $O ? P$ |
|------------|--------------|-------------|-------------|--------------------|---------|
| Full       | 1179         | 3082        | 499         | 2159               | 11      |
| MN + Reduc | 3768         | 2514        | 280         | 368                | 0       |

Predicates domain, we chose a generic domain since our interest lies in the trends within the logical entailment results between domain instances. Moreover, since our comparison study includes the *symbolic* component derived by the analyzer, the Predicates domain already includes highly precise information.

Each experiment uses subject programs consisting of 192 Java methods from previous research [6, 30], each exhibiting various levels of complexity. The total number of invariants is 6,930 compared in three different ablated comparison techniques. We used the Falcon [19] supercomputer and an existing DFA framework [5, 29] to compute all analyses.

#### 4.1 Comparable Domains

Table 1 shows the results of comparing Octagons ( $O$ ) against Zones ( $Z$ ) and Intervals ( $I$ ). Since the domains are ordered, an invariant of  $I$  or  $Z$  can only be as precise ( $\equiv O$ ) or less precise than Octagons ( $\prec O$ ). The table includes the counts of each comparison classification for each comparison technique.

The data shows that using full comparison, 78% of Octagon invariants are more precise than Interval invariants. However, using only minimal selection, the proportion reduces to 44%. In MN+Reduction, this proportion drops to 36%. A similar trend occurs when comparing octagonal invariants to zonal invariants: 26% to 11% to 6%, respectively.

#### 4.2 Incomparable Domains

Table 2 shows the comparison results between Symbolic Predicate and Octagon invariants. Since these domains are incomparable, the table contains three additional classifications:  $O \succ P$ , Octagons less precise than Symbolic Predicates, and  $O \prec\!\succ P$ , where both invariants are incomparable. A fifth column,  $O ? P$ , represents instances where our solver, Z3 [12], returned UNKNOWN. For this experiment, we used the iterative algorithm from previous work [4] for comparing incomparable domains and omit data for MN comparisons.

The data reveals that the number of equivalent invariants went up from 17% to 54%. The trends from other classifications are decreasing: from 44% to 36% for Symbolic Predicates more precise, from 7% to 4% for Octagons more precise, and from 31% to simply 5% for incomparable. Critically, minimal comparison resolved all 11 queries that previously returned UNKNOWN, a direct result of the reduced query size.

#### 4.3 Discussion

The data from the tables suggests that the spurious reduction technique and minimized comparison allows for more comprehensive comparison between abstract domains. That is, our reduced and minimal technique of comparison of invariants demonstrates a finer granularity of precision performance, especially for Symbolic Predicates. Evaluations show that the impact of Octagons on invariant precision may not be that significant. These findings substantiate arguments from previous work about weakly-relational domains computing predominately interval invariants [13, 14].

### 5 Related Work

Research presenting new techniques or new abstract domains tend to use one of two methods for comparison: either logical entailment [26, 30], or property or invariant capture [13, 17, 18, 23, 24]. Our work fits into the former group of work. However, this work extends previous work [3, 4] work by providing algorithms for identifying minimal subsets of Octagons.

In general, we notice an increasing amount of research in measuring and quantifying the differences between various abstract domains [3, 4, 8, 15, 16, 28]. While our work does not make any substantive arguments about distance between Intervals, Zones, and Octagons, we do present several techniques which we believe complement the current research.

### 6 Conclusion

In this work, we presented a reduction algorithm for Octagons that leverages the incremental nature of DFA. Combining with previous work on minimally comparing abstract domains, the reduction provides a more granular view of the precision benefits of Octagons compared to other numerical domains. Using real-world programs, we empirically evaluated the invariant precision of Octagons to Zones, Intervals, and Symbolic Predicates. The results show that a significant majority of invariants computed by Octagons are interval-valued, experimentally confirming anecdotal comments of other previous work [13].

As noted previously, the reduction algorithm combined with work from Larsen et al. [21, 22] would complement each other well in a model-checking context. Furthermore, an interesting future direction of this work would be exploring the interaction of the techniques for identifying minimal

changes within weakly-relational domains complements research attempting to create metrics around domain precision. Finally, it remains an open question whether the set of minimal changes can be used to improve the efficiency of DFA, by reducing the number of comparisons needed to determine subsumption or equality between Octagon states during the fixed point computation.

## Acknowledgments

This material is based upon work supported by the National Science Foundation under Grant No. CCF-19-42044. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

## References

- [1] Sven Apel, Dirk Beyer, Karlheinz Friedberger, Franco Raimondi, and Alexander von Rhein. 2013. Domain Types: Abstract-Domain Selection Based on Variable Usage. *Lecture Notes in Computer Science* (2013), 262–278. doi:10.1007/978-3-319-03077-7\_18
- [2] Kenny Ballou and Elena Sherman. 2022. Incremental Transitive Closure for Zonal Abstract Domain. In *NASA Formal Methods*. Jyotirmoy V. Deshmukh, Klaus Havelund, and Ivan Perez (Eds.). Springer International Publishing, Cham, 800–808. doi:10.1007/978-3-031-06773-0\_43
- [3] Kenny Ballou and Elena Sherman. 2023. Identifying Minimal Changes in the Zone Abstract Domain. In *Theoretical Aspects of Software Engineering*, Cristina David and Meng Sun (Eds.). Springer Nature Switzerland, Cham, 221–239. doi:10.1007/978-3-031-35257-7\_13
- [4] Kenny Ballou and Elena Sherman. 2023. Minimally Comparing Relational Abstract Domains. In *Automated Technology for Verification and Analysis*, Étienne André and Jun Sun (Eds.). Springer Nature Switzerland, Cham, 159–175. doi:10.1007/978-3-031-45332-8\_8
- [5] Kenny Ballou and Elena Sherman. 2025. *StaticCedTea*. doi:10.5281/ZENODO.16693791
- [6] Kenny Ballou and Elena Sherman. 2026. *Java Benchmark Artifacts*. doi:10.5281/ZENODO.18166810
- [7] Eric Bodden. 2018. Self-adaptive static analysis. *Proceedings of the 40th International Conference on Software Engineering: New Ideas and Emerging Results* (5 2018). doi:10.1145/3183399.3183401
- [8] Ignacio Casso, José F. Morales, Pedro López-García, Roberto Giacobazzi, and Manuel V. Hermenegildo. 2020. Computing Abstract Distances in Logic Programs. *Lecture Notes in Computer Science* (2020), 57–72. doi:10.1007/978-3-030-45260-5\_4
- [9] Aziem Chawdhary, Ed Robbins, and Andy King. 2018. Incrementally Closing Octagons. *Formal Methods in System Design* 54, 2 (1 2018), 232–277. doi:10.1007/s10703-017-0314-7
- [10] Christian Collberg, Ginger Myles, and Michael Stepp. 2007. An Empirical Study of Java Bytecode Programs. *Software: Practice and Experience* 37, 6 (2007), 581–641. doi:10.1002/spe.776
- [11] T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein. 2009. *Introduction to Algorithms*. McGraw-Hill.
- [12] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340. doi:10.1007/978-3-540-78800-3\_24
- [13] Graeme Gange, Zequn Ma, Jorge A. Navas, Peter Schachte, Harald Søndergaard, and Peter J. Stuckey. 2021. A Fresh Look at Zones and Octagons. *ACM Transactions on Programming Languages and Systems* 43, 3 (9 2021), 1–51. doi:10.1145/3457885
- [14] Graeme Gange, Jorge A. Navas, Peter Schachte, Harald Søndergaard, and Peter J. Stuckey. 2016. Exploiting Sparsity in Difference-Bound Matrices. *Lecture Notes in Computer Science* (2016), 189–211. doi:10.1007/978-3-662-53413-7\_10
- [15] Roberto Giacobazzi and Isabella Mastroeni. 2002. Domain Compression for Complete Abstractions. *Verification, Model Checking, and Abstract Interpretation* (12 2002), 146–160. doi:10.1007/3-540-36384-X\_14
- [16] Roberto Giacobazzi, Isabella Mastroeni, and Elia Perantoni. 2023. *How Fitting is Your Abstract Domain?* Springer Nature Switzerland, 286–309. doi:10.1007/978-3-031-44245-2\_14
- [17] Arie Gurfinkel and Sagar Chaki. 2010. Boxes: A Symbolic Abstract Domain of Boxes. *Lecture Notes in Computer Science* (2010), 287–303. doi:10.1007/978-3-642-15769-1\_18
- [18] Jacob M. Howe and Andy King. 2009. Logahedra: A New Weakly Relational Domain. *Lecture Notes in Computer Science* (2009), 306–320. doi:10.1007/978-3-642-04761-9\_23
- [19] Idaho C3+3 Collaboration. 2022. Falcon: High Performance Supercomputer. doi:10.7923/FALCON.ID
- [20] Jacques-Henri Jourdan. 2017. Sparsity Preserving Algorithms for Octagons. *Electronic Notes in Theoretical Computer Science* 331 (3 2017), 57–70. doi:10.1016/j.entcs.2017.02.004
- [21] K.G. Larsen, F. Larsson, P. Pettersson, and W. Yi. 1997. Efficient Verification of Real-Time Systems: Compact Data Structure and State-Space Reduction. In *Proceedings Real-Time Systems Symposium*. IEEE Comput. Soc, 14–24. doi:10.1109/real.1997.641265
- [22] Kim G. Larsen, Fredrik Larsson, Paul Pettersson, and Wang Yi. 2003. Compact Data Structures and State-Space Reduction for Model Checking Real-Time Systems. *Real-Time Systems* 25, 2/3 (2003), 255–275. doi:10.1023/a:1025132427497
- [23] Vincent Laviron and Francesco Logozzo. 2008. SubPolyhedra: A (More) Scalable Approach to Infer Linear Inequalities. *Verification, Model Checking, and Abstract Interpretation* (2008), 229–244. doi:10.1007/978-3-540-93900-9\_20
- [24] Francesco Logozzo and Manuel Fähndrich. 2010. Pentagons: A Weakly Relational Abstract Domain for the Efficient Validation of Array Accesses. *Science of Computer Programming* 75, 9 (9 2010), 796–807. doi:10.1016/j.scico.2009.04.004
- [25] Antoine Miné. 2001. A New Numerical Abstract Domain Based on Difference-Bound Matrices. *Lecture Notes in Computer Science* (2001), 155–172. doi:10.1007/3-540-44978-7\_10
- [26] Antoine Miné. 2004. Weakly Relational Numerical Abstract Domains. <https://pastel.archives-ouvertes.fr/tel-00136630>
- [27] Antoine Miné. 2006. The Octagon Abstract Domain. *Higher-Order and Symbolic Computation* 19, 1 (3 2006), 31–100. doi:10.1007/s10990-006-8609-1
- [28] Michael Schwarz, Simmo Saan, Helmut Seidl, Julian Erhard, and Vesal Vojdani. 2023. *Clustered Relational Thread-Modular Abstract Interpretation with Local Traces*. Springer Nature Switzerland, 28–58. doi:10.1007/978-3-031-30044-8\_2
- [29] Elena Sherman. 2018. Redesigning Soot’s Data-Flow Analysis Framework for Abstract Interpretation. In *Companion Proceedings for the ISSA/ECOOP 2018 Workshops* (Amsterdam, Netherlands) (ISSA ’18). Association for Computing Machinery, New York, NY, USA, 78–84. doi:10.1145/3236454.3236506
- [30] Elena Sherman and Matthew B. Dwyer. 2015. Exploiting Domain and Program Structure to Synthesize Efficient and Precise Data Flow Analyses (T). *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (11 2015). doi:10.1109/ase.2015.41
- [31] Ole Tange. 2024. *GNU Parallel 20240322 ('Sweden')*. doi:10.5281/zenodo.10901541 GNU Parallel is a general parallelizer to run multiple serial command line programs in parallel without changing them..

Received 2026-06-22; accepted 2026-08-04